

命令行井字棋游戏项目需求文档

一、项目概述

本项目是为了巩固黑马 Python 基础阶段的二维列表、函数封装、流程控制知识点设计的小游戏项目，比较有趣，不会枯燥，帮你把流程控制和数据容器的知识点练熟。

通过完成本项目，你可以练习二维列表、函数封装、循环判断、输入处理等核心基础知识点，帮你做到不需要查资料就能独立写小游戏的逻辑。

二、项目整体目录结构

这个项目比较小，不需要太多模块，你可以练习简单的模块划分：

代码块

```
1  tictactoe/                # 项目根目录
2  |—— main.py                # 主程序：游戏的交互入口
3  |—— game_tools.py         # 游戏的核心逻辑模块
```

三、功能需求

3.1 数据结构定义

用二维列表存储棋盘，比如：

代码块

```
1  # 初始棋盘
2  board = [
3      ['-', '-', '-'],
4      ['-', '-', '-'],
5      ['-', '-', '-'],
6  ]
7  # -表示空位置, X是玩家1, O是玩家2
```

练习二维列表的操作，比如 `board[row][col]` 访问指定位置的元素。

3.2 游戏核心函数 (game\_tools.py)

你需要封装以下函数，实现游戏的核心逻辑：

1. `print_board(board)`：打印棋盘，显示当前的棋盘状态，格式友好
 2. `check_move(board, row, col)`：检查落子是否合法，位置是否为空，是否在 0-2 的范围
 3. `make_move(board, row, col, player)`：落子，修改棋盘的对应位置
 4. `check_win(board)`：检查胜负，判断横、竖、斜是否有三个相同的棋子
 5. `check_full(board)`：检查棋盘是否满了，判断平局
-

四、主程序 `main.py` 需求

主程序是游戏的交互入口，用来练习**循环、判断、输入处理**的知识点：

4.1 程序流程

1. 初始化棋盘
2. 循环，轮流让玩家 1 (X) 和玩家 2 (O) 落子
3. 每次落子后，打印棋盘，检查胜负和平局
4. 如果有玩家赢了，或者平局，结束游戏

运行效果示例：

代码块

```
1  ===== 井字棋游戏 =====
2  当前棋盘：
3  - - -
4  - - -
5  - - -
6  玩家1(X)，请输入你要落子的行和列(0-2，用空格分隔)： 0 0
7  当前棋盘：
8  X - -
9  - - -
10 - - -
11 玩家2(O)，请输入你要落子的行和列(0-2，用空格分隔)： 1 1
12 ...
13 玩家1(X)赢了！游戏结束！
```

- 输入处理：如果用户输入的不是数字，或者超出范围，或者位置已经有棋子了，要提示输入无效，重新输入，不能让程序崩溃
-

五、知识点覆盖对照表

本项目完整覆盖黑马 Python 基础阶段的核心知识点，你可以对照检查：

黑马 Python 基础知识点	项目对应练习部分
环境搭建与 PyCharm 使用	项目目录创建、代码运行与调试
二维列表	棋盘的存储与操作
函数定义与调用	游戏逻辑的函数封装
循环与判断	游戏的循环、胜负判断
输入输出	玩家输入的处理、棋盘的打印
模块划分	把游戏逻辑放到单独的模块
异常处理	用户输入的错误处理

六、开发步骤指引

建议你按照以下顺序开发，循序渐进：

1. **第一步：搭建目录**：先创建好所有的空文件
2. **第二步：实现 game_tools**：先写游戏的核心函数，测试每个功能
3. **第三步：实现 main.py**：然后写主程序，把游戏的循环逻辑串起来
4. **第四步：整体测试**：测试整个游戏，测试落子、胜负判断、平局判断

七、验收标准

完成后你可以按照以下标准检查自己的项目是否达标：

1. 目录结构完全符合要求，模块可以正常导入，没有导入错误
2. 所有功能都可以正常运行，没有逻辑错误
3. 程序不会崩溃：不管用户输入什么错误内容，都只会给出友好提示，不会弹出 Traceback 报错
4. 可以正确判断胜负和平局
5. 所有基础知识点都有练习到，没有遗漏

（注：文档部分内容可能由 AI 生成）